Simulating Lattice Gauge Theories with Neural Networks

Sriram Bharadwaj: Sections 1 and 2*
Jack Isen: Section 5*
Seolhwa Kim: Section 3, 4, 6 and A*
Aditya Prasad Dash: Section B*

Physics 270M Final Project Report

Abstract

We investigate neural-network variational methods for the ground-state simulation of pure compact U(1) lattice gauge theory in $2+1$ dimensions, following the roadmap laid out by Luo et al [1]. Using the Kogut-Susskind Hamiltonian formulation, we construct a symmetry-aware ansatz for the ground-state wavefunction based on an equivariant complex-valued convolutional neural network. The architecture is designed to reflect the gauge and lattice symmetries of the theory by combining gauge-covariant link features with gauge-invariant plaquette features. The variational parameters are trained by minimizing the energy expectation value with variational Monte Carlo, while Metropolis-Hastings sampling is used to estimate observables with respect to the neural-network wavefunction. To provide an analytic benchmark, we also discuss the one-plaquette limit, which reduces to the Mathieu equation and yields a closed-form expression for the ground-state energy. Our study presents a concrete implementation of equivariant neural-network wavefunctions for lattice gauge theory and highlights their promise as scalable tools for nonperturbative Hamiltonian simulations.

*All authors contributed equally to the code.

Contents

1 Motivation **4**

1.1 Approaches to simulating quantum field theories . . . 4

1.2 Why a Neural Network ansatz? . . . 4

1.3 The scope of this work . . . 5

1.4 Outline of this paper . . . 5

1.5 The use of AI tools . . . 6

2 Setup: Kogut-Susskind Hamiltonian **6**

2.1 One-plaquette analytical solution as a benchmark . . . 7

3 Mapping Gauge Configurations to Neural Networks **8**

3.1 Overview of the architecture . . . 8

3.2 Trial wavefunctions from EquNN_U(1) . . . 8

3.3 Implementation in Python . . . 10

4 Getting the data and training with Variational Monte Carlo **13**

4.1 Variational principle . . . 13

4.2 Gradient of the variational energy . . . 14

4.3 Practical loss function . . . 15

4.4 Implementation of variational Monte Carlo . . . 16

4.5 Python implementation . . . 16

5 Results **18**

5.1 Model Training Results . . . 18

5.1.1 Loss functions . . . 18

5.1.2 Tuning Hyperparameters . . . 19

5.1.3 Comparison with Analytical Results . . . 19

5.2 Wavefunction Result . . . 22

6 Outlook **22**

6.1 Larger lattices and confinement observables . . . 22

6.2 Finite temperature and thermal states 23
6.3 Adding fermionic matter 23
6.4 Algorithmic improvements 23

A Details of code **26**
A.1 ComplexCNN 26
A.2 Metropolis-Hastings 26

B Model Architecture **28**
B.1 Constructing Gauge invariant objects 28
B.2 Lift Layers 29
B.3 Gauge Equivariant Layers 30
B.4 Gauge Invariant Layer 30

1 Motivation

The simulation of quantum field theories (QFTs) remains one of the central challenges in theoretical physics. These theories provide the fundamental language for describing both high-energy particle interactions and a wide range of emergent many-body phenomena, yet their nonperturbative regimes are often analytically inaccessible. In particular, strongly coupled gauge theories exhibit rich vacuum structure, confinement phenomena, and topological effects that are difficult to capture using conventional perturbative expansions. This makes the development of reliable nonperturbative methods an essential task [2, 3].

1.1 Approaches to simulating quantum field theories

A standard route to nonperturbative study is lattice discretization. By placing the theory on a spatial lattice, one obtains a regulated formulation with finitely many degrees of freedom, suitable in principle for numerical treatment [2, 3]. Much of the traditional success in this direction has come from Euclidean Monte Carlo methods, which are extremely powerful for equilibrium observables in sign-problem-free settings [3]. However, many physically interesting questions fall outside the natural scope of this framework, i.e. cases where the integral of the Lagrangian is purely real. Real-time dynamics, wavefunction structure, excited states, and theories with complex actions or finite density remain notoriously difficult to access [4, 5]. These limitations motivate the study of Hamiltonian formulations of lattice gauge theory, in which the problem is recast directly in terms of states, operators, and unitary time evolution [6, 5].

The Hamiltonian perspective is appealing for several reasons. First, it is naturally adapted to quantum dynamics and to questions about ground-state and excited-state wavefunction structure [6]. Second, it provides a direct bridge to quantum information language, where states are represented explicitly and observables can be computed in real time. Third, Hamiltonian lattice gauge theories are the natural starting point for digital and analog quantum simulation [5]. At the same time, however, the Hamiltonian approach comes with its own severe computational challenge: the Hilbert space grows *exponentially* with system size, even after removing gauge redundancy. As a result, brute-force methods such as exact diagonalization are restricted to very small lattices, and even tensor-network methods can face difficulties in higher dimensions or in regimes with substantial long-range entanglement [7].

1.2 Why a Neural Network ansatz?

This is precisely the setting in which machine learning approaches become attractive. Rather than representing the many-body wavefunction explicitly as a vector in Hilbert space, one seeks a compressed variational representation using a neural network ansatz. The network is trained to approximate the ground state or low-energy states by minimizing the energy expectation value, typically using variational Monte Carlo. The hope is that sufficiently expressive neural architectures can capture the essential structure of physically relevant states while using a number of parameters that scales more favorably than the full Hilbert-space dimension. In this sense, the machine learning

approach is not a replacement for the Hamiltonian formulation, but rather a new variational tool [8].

For gauge theories, however, generic neural-network ansätze are often poorly matched to the underlying structure of the problem. The physical Hilbert space is constrained by gauge invariance, and the dynamics respects geometric symmetries of the lattice such as translations, rotations, and reflections. If these symmetries are ignored, the model may waste expressive power learning features that are already known a priori, and training can become unnecessarily inefficient or unstable. This motivates the use of equivariant neural networks: architectures built so that their outputs transform in a controlled way under the symmetries of the theory. By incorporating gauge-theoretic and lattice symmetries directly into the ansatz, one can bias the variational search toward physically relevant states, improve sample efficiency, and obtain models whose internal structure is more closely aligned with the field-theoretic problem [9, 1].

1.3 The scope of this work

These notes are concerned with this program in the concrete setting of pure compact U(1) lattice gauge theory in 2 spatial dimensions and 1 time dimension, i.e. $(2+1)$ dimensions. This theory is simple enough to serve as a clean testing ground, yet rich enough to exhibit genuinely nontrivial gauge dynamics [6, 3]. In the Hamiltonian formulation, the basic degrees of freedom live on links of a two-dimensional spatial lattice, with electric-field operators and plaquette magnetic terms encoding the competition between electric and magnetic fluctuations [6]. Even in the absence of matter fields, the theory already poses a nontrivial variational problem because of gauge constraints and the large configuration space of link variables. It is therefore well-suited to study how symmetry-aware neural-network ansätze can be constructed [1]. Our goal will be to reproduce and analyze the results of [1].

The goal of the discussion is twofold. First, we want to explain why equivariant neural-network methods are a natural fit for Hamiltonian lattice gauge theory, both conceptually and computationally [9, 1]. Second, we want to describe in detail how these ideas are implemented for the pure U(1) theory in $2+1$ dimensions: what the degrees of freedom are, what symmetries must be respected, how the variational wavefunction is parameterized, and how observables are estimated in practice. In this way, the pure-gauge U(1) model serves as a minimal but meaningful example through which the broader strategy of machine-learning-based Hamiltonian simulation can be developed.

1.4 Outline of this paper

In this report, we have addressed the above questions in the following order. For convenience, we also provide a dictionary between the suggested paper template sections and our sections. As our project does not have data collection in the typical sense, certain suggested sections are grouped together or appear in a different order than suggested.

- Section 1: Motivation $\longleftrightarrow$ Abstract and Part 1: Big Picture
- Section 2: Setup $\longleftrightarrow$ Part 1: Big Picture

- Sections B: Architecture $\longleftrightarrow$ Part 5: Select Model and Train
- Section 4: Training Procedure $\longleftrightarrow$ Part 2: Get the Data and Part 4: Prepare the Data
- Section 5: Results $\longleftrightarrow$ Part 6: Fine Tune your Model and Part 7: Present your Solution
- Section 6 provides an outlook of the work and future directions

1.5 The use of AI tools

In this work, we have obtained guidance from ChatGPT in writing our code since the authors of [1] did not make their code or hyperparameter values openly available. Since our expertise on implementing lattice simulations is limited, we relied on ChatGPT to teach us standard tricks in using PyTorch. However, all of the code in the accompanying Jupyter notebook is written by us, except where indicated. We made every effort to ensure we understand all aspects of the code cells written by AI tools.

2 Setup: Kogut-Susskind Hamiltonian

In this paper, we consider a theory of pure $(2+1)$-dimensional electromagnetism described in the Kogut-Susskind Hamiltonian lattice gauge theory formulation. We work in Minkowski signature with metric conventions being $(+,-,-)$. Discretizing the theory on an $L \times L$ spatial lattice with unit-lattice-spacing, we may write down the Kogut-Susskind Hamiltonian for pure U(1) gauge theory at "gauge-coupling" $g^2 > 0$:

$$H = \frac{g^2}{2} \sum_{k=x,y} \sum_r E_{k,r}^2 + \frac{1}{g^2} \sum_r [1 - \cos(X_r + Y_{r+\hat{x}} - X_{r+\hat{y}} - Y_r)], \tag{1}$$

where the sum over $r = (x, y)$ runs over all lattice sites, i.e. $x, y \in \{0, \ldots, L-1\}$, and the fields (X_r, Y_r) and $E_{k,r}$ live on the links (see Fig. 1). Crucially the electric and magnetic link variables are canonically conjugate, i.e.

$$[E_{x,r}, e^{iX_r}] = e^{iX_r}, \qquad [E_y, e^{iY_r}] = e^{iY_r}. \tag{2}$$

The Hilbert space $\mathcal{H}$, prior to imposing gauge constraints, consists of many-body states of the form

$$\mathcal{H} = \bigotimes_r |X_r, Y_r\rangle . \tag{3}$$

Since the Hamiltonian in (1) is manifestly gauge-invariant, it only remains to impose Gauss' law on the states in $\mathcal{H}$. Since there are no sources or fermionic fields present, i.e. we deal with *pure* gauge theory, the electric charge density is simply $\rho = 0$, which implies that Gauss' law is the simple statement:

$$\nabla \cdot E\,|\psi\rangle = 0 \text{ for some } |\psi\rangle \in \mathcal{H}. \tag{4}$$

In particular, the subspace $\mathcal{H}_{\text{phys}} = \{|\psi\rangle \in \mathcal{H} \mid \nabla \cdot E\,|\psi\rangle = 0\} \subset \mathcal{H}$ is called the *physical* Hilbert space, since it consists of states that obey Gauss' law.

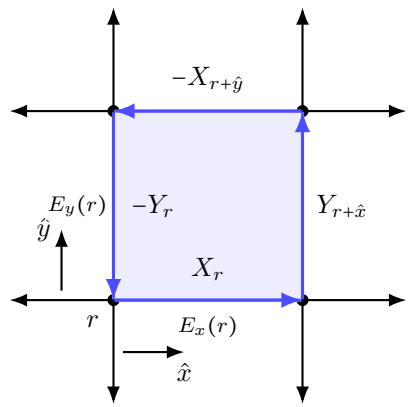

Figure 1: Single plaquette at site r illustrating the oriented plaquette angle $\varphi_r = X_r + Y_{r+\hat{x}} - X_{r+\hat{y}} - Y_r$ built from link variables (X_r, Y_r) with canonically conjugate electric fields $(E_{x,r}, E_{y,r})$.

2.1 One-plaquette analytical solution as a benchmark

In this subsection, we present a closed form analytical solution for the Hamiltonian (1). For the sake of the following discussion, it is convenient to switch to plaquette variables, i.e. $\varphi_r = X_r + Y_{r+\hat{x}} - X_{r+\hat{y}} - Y_r$.[1] Under this change of variables, the Hamiltonian (1) reduces to the following:

$$H_{1-\text{plaq}} = -2g^2 \frac{d^2}{d\varphi^2} + \frac{1}{g^2}(1 - \cos\varphi). \tag{5}$$

This yields the Schrödinger equation:

$$\left[\frac{d^2}{dz^2} + a - 2q\cos(2z)\right]\psi = 0, \qquad a = \frac{2}{g^2}\left(E - \frac{1}{g^2}\right), \qquad q = -\frac{1}{g^4}, \qquad z = \frac{\varphi}{2}. \tag{6}$$

This is simply the *Mathieu equation*, whose solutions are the so-called Mathieu functions. Importantly, the spectrum of the Mathieu equation is discrete, and the smallest eigenvalue is $a_0 = a_0(q)$. This gives a simple expression for the ground-state energy:

$$E_0(g^2) = \frac{g^2}{2} a_0\left(-\frac{1}{g^4}\right) + \frac{1}{g^2}. \tag{7}$$

This expression will provide a clean way to benchmark the performance of our neural network, since we can compare this one-plaquette energy to the energy density obtained from the neural network ansatz.[2]

[1] Note that the Gauss' law is automatically obeyed in the plaquette basis, which allows us to no longer worry about imposing this constraint explicitly.

[2] Details on the validity of this comparison may be found in [10], where a similar analysis is carried out for the complex Gaussian ansatz.

3 Mapping Gauge Configurations to Neural Networks

We denote a configuration of $U(1)$ link angles on the lattic by $\theta = \{\theta_l \mid l \in E\}$, where E is the set of links, and by α the collection of trainable parameters in the neural network.[3]

3.1 Overview of the architecture

Our goal is to construct a variational ansatz for the ground-state wavefunction,

$$\psi_\alpha(\theta) \tag{8}$$

using a neural network whose structure reflects the gauge-theoretic content of the problem. The overall procedure is as follows.

1. Starting from a lattice configuration of $U(1)$ link variables, we use a gauge-aware neural network, denoted **EquNN_U(1)**, to produce a trial wavefunction $\psi_\alpha(\theta)$.
2. For fixed network parameters α, we use **Metropolis–Hastings** sampling to draw a batch of configurations θ from the probability distribution

   $$p_\alpha(\theta) \propto |\psi_\alpha(\theta)|^2 \tag{9}$$

   These samples are then used to estimate observables, in particular the expectation value of the Hamiltonian.
3. We then update the network parameters using **variational Monte Carlo** (VMC), i.e. we optimize α so as to lower the sampled estimate of the energy expectation value

   $$\frac{\langle \psi_\alpha | H | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} \tag{10}$$

In this way, Metropolis–Hastings is used to sample configurations from the current variational state, while VMC is the step that improves the variational state itself.

3.2 Trial wavefunctions from EquNN_U(1)

The equivariant neural network maps a link configuration to a variational wavefunction

$$\psi_\alpha(\theta) \tag{11}$$

where the link variables are written in phase form as

$$\phi = \{\phi_l = e^{i\theta_l} \mid l \in E\}. \tag{12}$$

[3] Parts 3 (preparing the data) and 4 (exploring the data) of the suggested template do not apply directly to our project, since we do not work with an experimental dataset. Instead, our configurations are generated dynamically during training via Metropolis–Hastings sampling, as explained in Sections 3 and 4.

At a conceptual level, the construction has two parts:

1. an *equivariant block*, which acts on gauge-covariant link features, and
2. an *invariant head*, which acts on gauge-invariant plaquette features and produces the final scalar output.

More explicitly, the network consists of three stages: a lift, a sequence of equivariant blocks, and an invariant head. Schematically,

$$
\begin{aligned}
\phi = \{\phi_l \mid l \in E\} \quad & \xrightarrow{\text{Lift}} \quad \left\{ \vec{f}^{(0)}(\phi_l),\ \vec{g}_p^{(0)} = \prod_{l \in p} \vec{f}^{(0)}(\phi_l) \;\middle|\; l \in E,\ p \in P \right\} \\
& \longrightarrow \left\{ \vec{f}^{(1)}(\phi_l),\ \vec{g}_p^{(1)} \;\middle|\; l \in E,\ p \in P \right\} \\
\text{Equivariant block} \quad & \vdots \\
& \longrightarrow \left\{ \vec{f}^{(N)}(\phi_l),\ \vec{g}_p^{(N)} \;\middle|\; l \in E,\ p \in P \right\} \\
\text{Invariant head} \quad & \xrightarrow{\text{Avg. pooling}} \quad \vec{\psi} = \frac{1}{|P|} \sum_{p \in P} \vec{h}^{\text{inv}}\left(\vec{g}_p^{(N)}\right) \\
& \xrightarrow{\text{Linear map}} \quad \log \psi_\alpha(\phi) = \vec{c} \cdot \vec{\psi} \in \mathbb{R}
\end{aligned}
\tag{13}
$$

Here P denotes the set of plaquettes. The lift first maps each link variable into an F-dimensional complex feature space,

$$
\begin{aligned}
\text{Lift}: \ U(1) &\to \mathbb{C}^F \\
\phi_l &\mapsto \vec{f}(\phi_l)
\end{aligned}
\tag{14}
$$

Thus, each link is assigned a complex feature vector.

These lifted link features are then processed through N equivariant layers. At layer k, we first form a plaquette feature

$$
\vec{g}_p^{(k)} = \prod_{l \in p} \vec{f}^{(k)}(\phi_l) \tag{15}
$$

and then update each link feature according to

$$
\vec{f}^{(k+1)}(\phi_l) = \vec{f}^{(k)}(\phi_l) + \vec{f}^{(k)}(\phi_l) \diamond \vec{h}_{x,y}^{(k)}\left(\vec{g}_p^{(k)}\right) \tag{16}
$$

where $\diamond$ denotes component-wise multiplication, and

$$
\vec{h}_{x,y}^{(k)} : \mathbb{C}^F \to \mathbb{C}^F \oplus \mathbb{C}^F \tag{17}
$$

is a complex-valued convolutional map. In practice, the nonlinearity is taken to be the complex ELU activation,

$$
\begin{aligned}
\text{cELU}(z) &= \text{ELU}(\text{Re}\, z) + i\ \text{ELU}(\text{Im}\, z), \\
\text{ELU}(x) &= \begin{cases} x, & x > 0 \\ e^x - 1, & x \le 0 \end{cases}
\end{aligned}
\tag{18}
$$

The role of the equivariant block is to update the link features in a way that remains compatible with the gauge structure encoded by the plaquette variables. [SK: What do you mean by gauge structure? My understanding is that it is a version of gauge transformation on the feature vector as I wrote originally.] After N such layers, the resulting plaquette features are fed into an invariant head,

$$\vec{h}^{\text{inv}} : \mathbb{C}^F \to \mathbb{C}^F \tag{19}$$

followed by spatial averaging over plaquettes and a final linear projection. This produces a real number $\log \psi_\alpha(\phi)$, which defines the variational ansatz. Thus, while intermediate representations are complex-valued, the final network output is a real scalar.

3.3 Implementation in Python

We now explain how the architecture above is implemented in Python. We discuss the equivariant block, the invariant head, and the full assembly in turn.

Equivariant block

```python
class EquivariantBlock(nn.Module):
    def __init__(self, Ffeat, kernel_size=3, cnn_layers=2, hidden=None):
        super().__init__()
        hidden = hidden or Ffeat
        self.cnn = ComplexCNN(in_ch=Ffeat, hidden_ch=hidden, out_ch=2*Ffeat,
                              kernel_size=kernel_size, n_layers=cnn_layers)
        self.act = cELU()

    def forward(self, E):
        # E: (B,F,2,H,W) complex
        # Split Ex, Ey
        Ex = E[:, :, 0]  # (B,F,H,W)
        Ey = E[:, :, 1]  # (B,F,H,W)

        # Compute per-feature plaquette loop P = Ex * Ey(x+1) * Ex(y+1)^* * Ey^*
        P = Ex * shift_x_fwd(Ey) * torch.conj(shift_y_fwd(Ex)) * torch.conj(Ey)  # (B,F,H,W)

        # Apply CNN to P to get M of shape (B,2F,H,W), then reshape to (B,F,2,H,W)
        M = self.cnn(P)  # (B,2F,H,W)
        B, _, H, W = M.shape
        Ffeat = E.shape[1]
        M = M.view(B, Ffeat, 2, H, W)

        # Return equivariant update with residual: E + E * cELU(M)
        return E + E * self.act(M)
```

The tensor

$$E \in \mathbb{C}^{B\times F\times 2\times H\times W} \tag{20}$$

stores the link features. Its indices have the following meaning:

$$\begin{aligned} B &: \text{ batch size, i.e. the number of sampled } U(1) \text{ configurations,} \\ F &: \text{ feature dimension,} \\ 2 &: \text{ link direction } (x \text{ or } y), \\ H, W &: \text{ spatial lattice dimensions.} \end{aligned} \tag{21}$$

The forward pass proceeds as follows.

1. The link features are split into their x- and y-directed parts, E_x and E_y.
2. From these, the code constructs a plaquette feature tensor

$$P = E_x \cdot \mathrm{shift}_x(E_y) \cdot \mathrm{shift}_y(E_x)^* \cdot E_y^* \tag{22}$$

   which is the feature-space analogue of an oriented plaquette product.
3. This plaquette tensor is fed into a complex CNN that maps

$$\mathbb{C}^F \to \mathbb{C}^{2F} \tag{23}$$

   The factor of 2 reflects the fact that the output is later split into updates for the x- and y-directed link features.
4. Finally, the output is reshaped and inserted into the residual update

$$E \mapsto E + E \cdot \mathrm{cELU}(M) \tag{24}$$

   which implements the equivariant map in Eq. (16).

Invariant head

```python
class InvariantHead(nn.Module):
    def __init__(self, Ffeat, inv_features=4, kernel_size=3, inv_layers=2):
        super().__init__()
        self.cnn = ComplexCNN(in_ch=Ffeat, hidden_ch=inv_features, out_ch=inv_features,
                              kernel_size=kernel_size, n_layers=inv_layers)
        self.lin = nn.Linear(2*inv_features, 1)

    def forward(self, E):
        # Build plaquette loops P from E (same as in EquivariantBlock)
        Ex = E[:, :, 0]
        Ey = E[:, :, 1]
```

```python
        P = Ex * shift_x_fwd(Ey) * torch.conj(shift_y_fwd(Ex)) * torch.conj(Ey)  # (B,F,H,W)

        # Run CNN, average pool over space
        Z = self.cnn(P)                 # (B,inv_features,H,W) complex
        zbar = Z.mean(dim=(-2, -1))     # (B,inv_features) complex

        # Concat real and imag -> linear -> (B,)
        feat = torch.cat([zbar.real, zbar.imag], dim=-1)  # (B,2*inv_features)
        logpsi = self.lin(feat).squeeze(-1)
        return logpsi
```

The invariant head again begins by constructing plaquette features from the link-feature tensor E. These plaquette features are then processed by a complex CNN implementing the map $\vec{h}^{\text{inv}}$. The result is average-pooled over the spatial lattice, producing a single complex feature vector for each sampled configuration. Finally, the real and imaginary parts are concatenated and passed through a linear layer to produce

$$\log \psi_\alpha(\phi) \tag{25}$$

Thus, the invariant head converts a spatially distributed set of gauge-invariant plaquette features into a single scalar output suitable for defining the variational wavefunction.

Lift and assembly

```python
class EquNN_U1(nn.Module):
    def __init__(self, L, eq_layers=2, eq_features=2,
                 kernel_size=3, eq_cnn_layers=2,
                 inv_features=4, inv_layers=2):
        super().__init__()
        self.L = int(L)
        self.eq_features = int(eq_features)

        # Direction-preserving lift
        self.lift_x = ComplexConv2d(in_ch=1, out_ch=self.eq_features, kernel_size=1, bias=True)
        self.lift_y = ComplexConv2d(in_ch=1, out_ch=self.eq_features, kernel_size=1, bias=True)

        # Equivariant blocks
        self.eq_blocks = nn.ModuleList([
            EquivariantBlock(self.eq_features, kernel_size=kernel_size, cnn_layers=eq_cnn_layers)
            for _ in range(eq_layers)
        ])

        # Invariant head
        self.head = InvariantHead(self.eq_features, inv_features=inv_features,
```

```
                                kernel_size=kernel_size, inv_layers=inv_layers)

    def forward(self, theta):
        # theta: (B,2,L,L) real
        U = link_phases(theta)          # (B,2,L,L)
        Ux = U[:, 0:1]                  # (B,1,L,L)
        Uy = U[:, 1:2]                  # (B,1,L,L)

        # lift separately
        Ex = self.lift_x(Ux)            # (B,F,L,L)
        Ey = self.lift_y(Uy)            # (B,F,L,L)

        # pack as link-like features
        E = torch.stack([Ex, Ey], dim=2)  # (B,F,2,L,L)

        for blk in self.eq_blocks:
            E = blk(E)

        return self.head(E)             # (B,) real logpsi
```

The class `EquNN_U1` assembles the full variational ansatz. The input is a real tensor of link angles,

$$\theta \in \mathbb{R}^{B\times 2\times L\times L} \tag{26}$$

These angles are first mapped to link phases $U = e^{i\theta}$. The x- and y-directed links are then lifted separately into F-dimensional complex feature spaces using one-channel complex convolutions. This produces the tensors E_x and E_y, which are stacked into the link-feature tensor

$$E \in \mathbb{C}^{B\times F\times 2\times L\times L} \tag{27}$$

The tensor E is then passed sequentially through the list of equivariant blocks, after which the invariant head produces the final scalar output. In this way, `EquNN_U1` realizes the full map

$$\theta \mapsto \log \psi_\alpha(\theta) \tag{28}$$

4 Getting the data and training with Variational Monte Carlo

4.1 Variational principle

To approximate the ground state, we consider a family of trial wavefunctions

$$\psi_\alpha(\theta) = \langle \theta | \psi_\alpha \rangle \tag{29}$$

parameterized by the neural-network weights α. We then optimize these parameters using the variational principle:

$$\overline{\alpha} = \arg\min_{\alpha} \frac{\langle \psi_\alpha | \hat{H} | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} \tag{30}$$

Although the class of wavefunctions realizable by the CNN does not span the full Hilbert space,

$$\{\psi_\alpha(\theta) \mid \alpha : \text{weights and biases of neural network}\} \tag{31}$$

it may nevertheless contain good approximations to the true ground state. Our objective is therefore to find the optimal parameter set $\overline{\alpha}$ within this restricted variational family.

4.2 Gradient of the variational energy

To minimize the energy expectation value in Eq. (30), we compute its gradient with respect to the network parameters:

$$\nabla_\alpha \left[\frac{\langle \psi_\alpha | \hat{H} | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} \right] \tag{32}$$

Using standard manipulations, one finds

$$\nabla_\alpha \left[\frac{\langle \psi_\alpha | \hat{H} | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} \right] = \frac{\langle \nabla_\alpha \psi_\alpha | \hat{H} | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} - \frac{\langle \psi_\alpha | \hat{H} | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} \frac{\langle \nabla_\alpha \psi_\alpha | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} + \text{h.c.} \tag{33}$$

Next, using

$$\nabla_\alpha \psi_\alpha(\theta) = \psi_\alpha(\theta)\, \nabla_\alpha \log \psi_\alpha(\theta) \tag{34}$$

we rewrite the first term as

$$\begin{aligned} \frac{\langle \nabla_\alpha \psi_\alpha | \hat{H} | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} &= \frac{\int \mathrm{d}\theta\, \nabla_\alpha \psi_\alpha^*(\theta)\, \hat{H} \psi_\alpha(\theta)}{\int \mathrm{d}\theta\, |\psi_\alpha(\theta)|^2} \\ &= \frac{\int \mathrm{d}\theta \left(\nabla_\alpha \log \psi_\alpha^*(\theta) \right) \psi_\alpha^*(\theta)\, E_{\mathrm{loc}}(\theta)\, \psi_\alpha(\theta)}{\int \mathrm{d}\theta\, |\psi_\alpha(\theta)|^2} \\ &= \langle\!\langle \nabla_\alpha \log \psi_\alpha^*(\theta)\, E_{\mathrm{loc}}(\theta) \rangle\!\rangle \end{aligned} \tag{35}$$

where

$$E_{\mathrm{loc}}(\theta) = \frac{\hat{H} \psi_\alpha(\theta)}{\psi_\alpha(\theta)} \tag{36}$$

is the local energy, and

$$\mathrm{d}\theta = \prod_{l \in E} \mathrm{d}\theta_l \tag{37}$$

Here $\langle\!\langle \cdot \rangle\!\rangle$ denotes expectation with respect to the probability distribution

$$p_\alpha(\theta) \propto |\psi_\alpha(\theta)|^2 \tag{38}$$

which we estimate numerically using a finite batch of sampled configurations.

Similarly,

$$\begin{aligned}\frac{\langle \nabla_\alpha \psi_\alpha | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} &= \frac{\int \mathrm{d}\theta \, \nabla_\alpha \log \psi_\alpha^*(\theta) \, |\psi_\alpha(\theta)|^2}{\int \mathrm{d}\theta \, |\psi_\alpha(\theta)|^2} \\ &= \langle\!\langle \nabla_\alpha \log \psi_\alpha^*(\theta) \rangle\!\rangle\end{aligned} \tag{39}$$

and

$$\begin{aligned}\frac{\langle \psi_\alpha | \hat{H} | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} &= \frac{\int \mathrm{d}\theta \, \psi_\alpha^*(\theta) \hat{H} \psi_\alpha(\theta)}{\int \mathrm{d}\theta \, |\psi_\alpha(\theta)|^2} \\ &= \frac{\int \mathrm{d}\theta \, E_{\mathrm{loc}}(\theta) \, |\psi_\alpha(\theta)|^2}{\int \mathrm{d}\theta \, |\psi_\alpha(\theta)|^2} \\ &= \langle\!\langle E_{\mathrm{loc}}(\theta) \rangle\!\rangle\end{aligned} \tag{40}$$

Substituting Eqs. (35)–(40) into Eq. (33), we obtain

$$\begin{aligned}\nabla_\alpha \left[\frac{\langle \psi_\alpha | \hat{H} | \psi_\alpha \rangle}{\langle \psi_\alpha | \psi_\alpha \rangle} \right] &= \langle\!\langle E_{\mathrm{loc}}(\theta) \, \nabla_\alpha \log \psi_\alpha^*(\theta) \rangle\!\rangle \\ &\quad - \langle\!\langle E_{\mathrm{loc}}(\theta) \rangle\!\rangle \, \langle\!\langle \nabla_\alpha \log \psi_\alpha^*(\theta) \rangle\!\rangle + \mathrm{h.c.}\end{aligned} \tag{41}$$

4.3 Practical loss function

In the implementation, the local energy is computed inside a `torch.no_grad()` block:

```
with torch.no_grad():
    E_loc = local_energy_fd(
        theta,
        model,
        g2=g2,
        mag_prefactor=mag_prefactor,
        eps=eps_fd,
        chunk_links=chunk_links
    )
    E_mean = E_loc.mean()
```

This means that, during backpropagation, the optimizer does not differentiate through the numerical evaluation of E_{loc}. Denoting this detached quantity by $\tilde{E}_{\mathrm{loc}}$, the optimization is implemented through the stochastic loss

$$\mathrm{loss} = \left\langle\!\!\left\langle \left[\tilde{E}_{\mathrm{loc}}(\theta) - \langle \tilde{E}_{\mathrm{loc}}(\theta) \rangle \right] \log \psi_\alpha(\theta) \right\rangle\!\!\right\rangle \tag{42}$$

whose gradient reproduces the desired variational-energy gradient up to the same Monte Carlo approximation.

4.4 Implementation of variational Monte Carlo

For fixed parameters α, we sample a batch of configurations

$$\theta_1, \ldots, \theta_B \tag{43}$$

from the probability distribution

$$p_\alpha(\theta) \propto |\psi_\alpha(\theta)|^2 \tag{44}$$

using the Metropolis–Hastings algorithm.[4][5]

For each sampled configuration, we compute the local energy

$$E_{\text{loc}}(\theta) = \frac{\hat{H}\psi_\alpha(\theta)}{\psi_\alpha(\theta)}, \tag{45}$$

and then estimate the loss in Eq. (42). The Adam optimizer is used to backpropagate through $\log \psi_\alpha(\theta)$ and update the parameters,

$$\alpha_0 \to \alpha_1 \to \alpha_2 \to \cdots. \tag{46}$$

After each update, the probability distribution changes because the wavefunction itself has changed. We therefore resample configurations from the updated distribution and repeat the procedure for n_{iter} iterations.

4.5 Python implementation

```python
def train_vmc_equnn(
    L=2, g2=1.0, mag_prefactor=1.0,
    n_iters=200, batch=32,
    burn_sweeps=30, between_sweeps=5,
    mh_step_size=0.25, lr=2e-3,
    eps_fd=1e-2, chunk_links=8, seed=0
):
    torch.manual_seed(seed)
    np.random.seed(seed)

    model = EquNN_U1(L=L).to(device=device, dtype=dtype)
    opt = torch.optim.Adam(model.parameters(), lr=lr)

    theta = (TWOPI * torch.rand(batch, 2, L, L, device=device, dtype=dtype)) % TWOPI
```

[4]Given a proposed configuration θ', the Metropolis–Hastings acceptance probability is

$$\min\left(1, \frac{|\psi_\alpha(\theta')|^2}{|\psi_\alpha(\theta)|^2}\right)$$

The proposal is accepted whenever this quantity exceeds a uniformly drawn random number in $[0, 1)$.

[5]All sampled configurations in a given batch are drawn from the same variational wavefunction, i.e. they share the same parameter set α.

```python
    losses = []

    # burn-in
    theta, acc = mh_sample(model, theta, n_sweeps=burn_sweeps, step_size=mh_step_size)

    for it in range(n_iters):
        theta, acc = mh_sample(model, theta, n_sweeps=between_sweeps,
         ↪  step_size=mh_step_size)

        with torch.no_grad():
            E_loc = local_energy_fd(theta, model, g2=g2, mag_prefactor=mag_prefactor,
                                    eps=eps_fd, chunk_links=chunk_links)
            E_mean = E_loc.mean()

        logpsi = model(theta)  # grads flow here
        loss = 2.0 * torch.mean((E_loc - E_mean) * logpsi)

        opt.zero_grad(set_to_none=True)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
        losses.append(loss)
        if it % 50 == 0 or it == n_iters - 1:
            print(f"g^2={g2:.1f}  it={it:4d}  <E>={float(E_mean): .6f}  acc={acc:.3f}")

    return model
```

The logic of this code is as follows.

1. A model `EquNN_U1` is initialized, together with an Adam optimizer.
2. An initial batch of random link-angle configurations is generated.
3. A burn-in phase is performed using Metropolis–Hastings so that the batch better reflects the distribution induced by the current wavefunction.
4. At each training step, the batch is refreshed by several additional Metropolis–Hastings sweeps.
5. The local energy and its batch mean are computed with gradients detached.
6. The quantity `logpsi = model(theta)` is evaluated with gradients enabled, and the loss

$$2\,\mathrm{mean}\big[(E_{\mathrm{loc}} - E_{\mathrm{mean}})\log\psi\big] \tag{47}$$

   is backpropagated.
7. The optimizer then updates the network parameters, producing a new variational state for the next iteration.

In summary, the training loop alternates between two steps: sampling configurations from the current trial state and updating the trial state using those samples. This is precisely the variational Monte Carlo procedure specialized to our equivariant neural-network ansatz.

5 Results

In this section, we present our results in two different contexts. First, we look at internal performance. We show that our model performs well at strong couplings in terms of the loss function, while it converges much more slowly at weak couplings. We give a few possible explanations for the discrepancy at different couplings. Second, we compare our model to expectations from theory, and to competing models in the literature. We find that our model performs as expected when compared to theoretical results.

5.1 Model Training Results

Recall from the previous section that the Equivariant Neural Network model is trained in the following way: First, a lattice configuration is randomly generated and fed into the model, which outputs a wavefunction. Then, the Metropolis-Hastings algorithm is used to find a set of highly probable lattice configurations. With these probable lattice configurations, the local and mean energies are then calculated from the wavefunction. These are used to find the loss function (42) by computing weighted averages over the configurations[6]. The Adam optimizer then tunes the model parameters, changing $\psi(\theta)$. With this in mind, it is clear that there are two distinct notions of convergence. First, there is convergence of the Metropolis-Hastings algorithm, i.e. convergence to the lattice configurations of maximal probability. Second, there is convergence of the Variational Monte Carlo, i.e. convergence of the wavefunction itself. It will be the second notion of convergence that we are primarily concerned with.

5.1.1 Loss functions

The loss functions for various couplings are presented in the figure above. As can be easily seen, convergence is very rapid when $g^2 \gg 1$. Moreover, the final loss after training is $\ll 1$ in all cases, but it is significantly smaller at large couplings. Additionally, for smaller couplings, convergence takes much longer than for large couplings. In the plot above, the model was trained for 2000 epochs, and the final loss is still significantly larger than the loss for large coupling models. It is worth noting here that our results differ from the performance of the EquNN model in [1], which finds excellent convergence in both the large and small coupling regimes. As such, it is clear that our implementation of the EquNN could be improved. The source of this issue is not immediately clear, but there are two likely culprits. First, the Metropolis-Hastings algorithm, in its current

[6] Our implementation of the Metropolis-Hastings algorithm simply selects the most probable states, which is justified when the probability distribution is sharply peaked. A more delicate approach is to first perform a short "burn-in", discarding initial samples and moving away from highly improbable states, and then record and average over all subsequent lattice configurations.

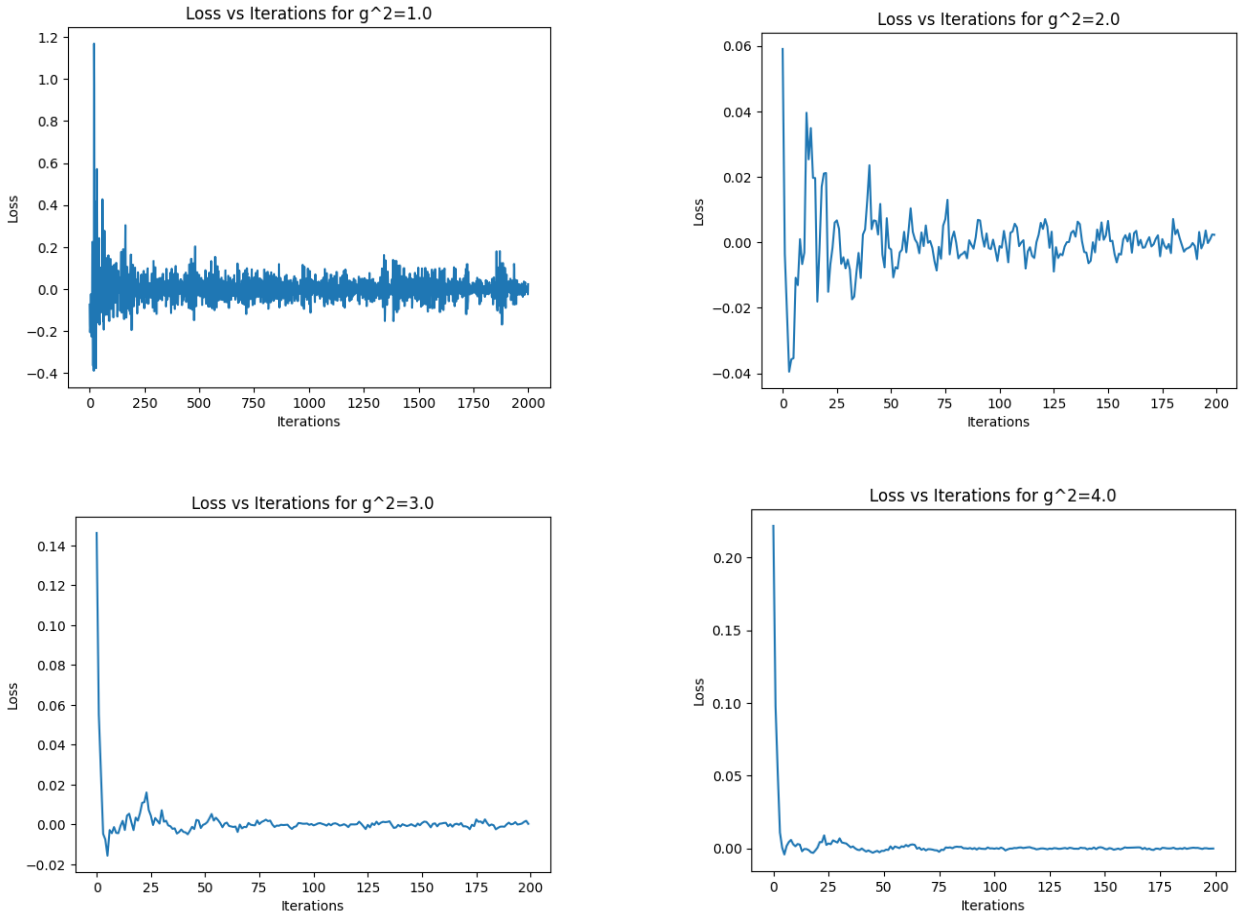

Figure 2: Loss functions for various values of g^2

implementation, is missing information about lower probability field configurations, which may be more important at weak couplings. Second, the equivariant layer may require more free parameters in order to converge at weak coupling, where gauge invariance is more subtle. The true explanation is likely a combination of these two factors.

5.1.2 Tuning Hyperparameters

Below is a figure showing the final loss function after training for various learning rates in the $g^2 = 1$ model. The $g^2 = 1$ model is unique among the EquNN models that we tested, in that it converges slowly for any learning rate (see figure 3). The models with $g^2 > 1$ were all trained at a learning rate of 1×10^{-3}, which resulted in rapid convergence.

5.1.3 Comparison with Analytical Results

In section 2, we derived an analytical solution for the ground state energy of a single plaquette. Our model runs on an $L \times L$ lattice, with $L \geq 2$. Hence, our model has at least 4 plaquettes. Naively, it would seem that we cannot directly compare our model's results to these theoretical predictions. However, if we compare the energy *density* between our model's predicted ground state and the one

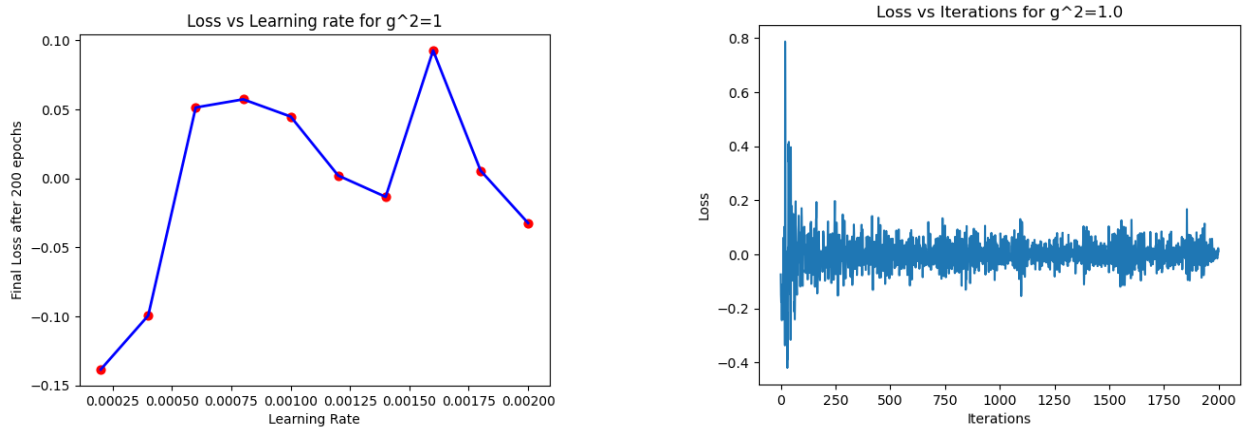

Figure 3: (Left) Final loss after 200 iterations versus learning rate. The left panel seems to suggest that there is a learning rate (1.4×10^{-3}) which results in minimal absolute value of the loss. (Right) Loss as a function of iterations for this "optimal" learning rate. We can see that despite choosing this optimal rate, convergence does not occur rapidly.

plaquette analytical solution, there is a regime where the two should agree.

Specifically, results [10] suggest that at large couplings, the energy is minimized when all plaquettes align. This means that every plaquette *individually* minimizes its own energy, and the Mathieu function solution is valid.

We calculate the energy using the code below, which takes in an already trained model, runs the Metropolis Hastings algorithm to find the maximum probability configuration, and then calculates the mean and standard deviation of the local energy (the energy computed at each plaquette). Note, this local energy is then divided by the number of plaquettes so that it can be directly compared to the analytical solution's energy.

Estimate Energy

```python
@torch.no_grad()
def estimate_energy(
    model, L, g2, mag_prefactor=1.0,
    batch=256, burn_sweeps=80, meas_sweeps=40,
    mh_step_size=0.25, eps_fd=1e-2, chunk_links=8
):
    theta = (TWOPI * torch.rand(batch, 2, L, L, device=device, dtype=dtype)) % TWOPI
    theta, _ = mh_sample(model, theta, n_sweeps=burn_sweeps, step_size=mh_step_size)
    theta, acc = mh_sample(model, theta, n_sweeps=meas_sweeps, step_size=mh_step_size)

    E_loc = local_energy_fd(theta, model, g2=g2, mag_prefactor=mag_prefactor,
                            eps=eps_fd, chunk_links=chunk_links)

    mean = float(E_loc.mean().item())
    std  = float(E_loc.std().item())
    sem  = std / np.sqrt(batch)
```

```
    return mean, sem, acc, theta
```

This energy can be further subdivided into an electric/kinetic and a magnetic energy. Below, we present the expectation value of ground state energy density, magnetic energy density, kinetic energy density, and the analytical one-plaquette ground state energy density, all for the 2×2 lattice.

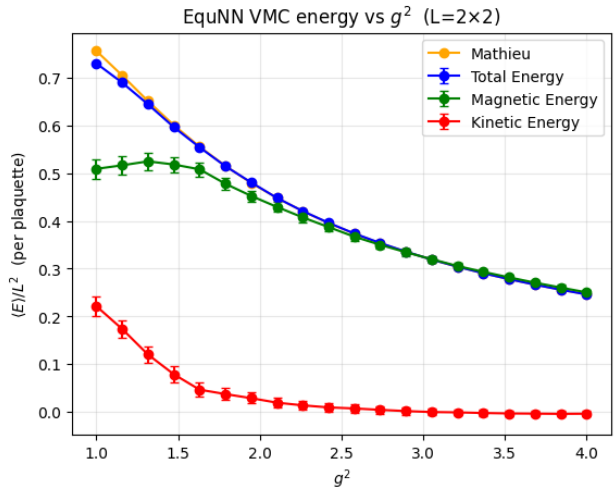

Figure 4: Energy Comparison for 2×2 Lattice

As expected from the discussion above, the ground state energy density predicted by our model does not match the Mathieu (analytical) energy density at weak couplings, but does match it at strong couplings. Additionally, at weak couplings, the uncertainties in the magnetic and kinetic energies are much higher than at large couplings. We can see that at weaker couplings, the kinetic energy becomes a larger fraction of the total energy, as expected. Additionally, we include a plot of the energy difference between the Mathieu and model energies. We can see that at strong couplings, the percent difference is well below 0.1%.

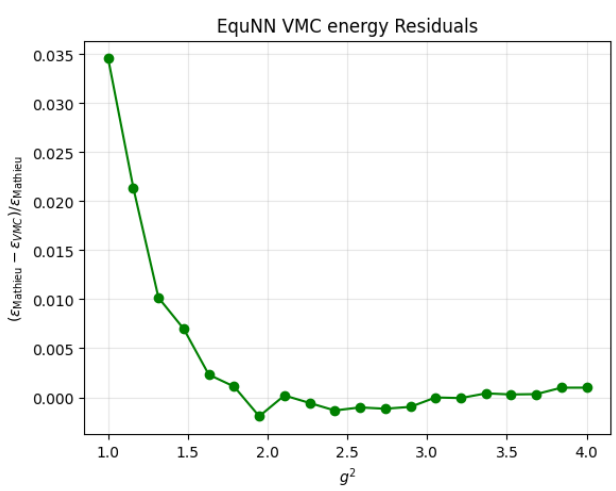

Figure 5: Difference in Energy Densities between Analytical and Predicted Ground States

5.2 Wavefunction Result

Finally, we present a visualization of the wavefunction for the $g^2 = 4$ model and compare it to the ground state analytical wavefunction.

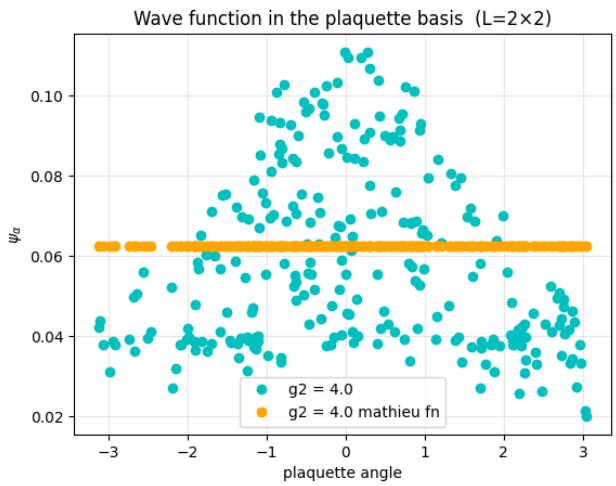

Figure 6: Wavefunctions of the model and analytical solution for $g^2 = 4$

As we can see, the wavefunctions for the two solutions look nothing alike. Of course, this is what we expect. Firstly, the neural network solution lives on a 2×2 lattice, so it is likely multi-valued. The wavefunction depends on multiple angles, rather than just one. Additionally, an approximate ground state found via the variational principle is not expected to have a similar wavefunction to the true ground state unless a very good initial ansatz was made, even if the energy is approximately the ground state energy. Our model, composed of complex ReLu functions, is likely not complicated enough to approximate the true ground state.

6 Outlook

There are several natural extensions of the present work. Since our current implementation focuses on pure compact $U(1)$ gauge theory on small lattices at zero temperature, the most immediate next steps are to study larger lattices, finite temperature, and dynamical fermions, while also improving the variational ansatz and optimization procedure.

6.1 Larger lattices and confinement observables

A first extension is to study larger spatial lattices and directly probe confinement through Wilson loops. For a closed loop γ enclosing a surface $\mathrm{Int}(\gamma)$, the Wilson loop operator is

$$W(\gamma) = \exp\left\{ i \int_{\mathrm{Int}(\gamma)} \vec{B} \cdot d\vec{S} \right\}. \tag{48}$$

In a confining regime, large Wilson loops obey an area law, while in a deconfined regime they instead exhibit perimeter-law behavior [2, 3, 11]. Since compact $U(1)$ gauge theory in $(2+1)$ dimensions is confining at zero temperature, this structure should become more apparent as the lattice size is increased and larger loops become accessible [3, 12, 1].

This would provide a significantly stronger test than the ground-state energy alone. A natural next step is therefore to train the model on a sequence of lattice sizes and compute

$$\langle 0|W(\gamma)|0\rangle \propto e^{-T\mathrm{Area}(\gamma)} \tag{49}$$

for rectangular contours γ of varying area and perimeter, with the goal of extracting an effective string tension T from the large-loop behavior. This is especially well motivated by the results of [1], where clean confinement diagnostics were obtained on lattices as large as 14×14.

6.2 Finite temperature and thermal states

A second extension is to study the theory at nonzero temperature. Compact three-dimensional $U(1)$ lattice gauge theory is known to exhibit a finite-temperature deconfinement transition, with evidence that its critical behavior is related to that of the two-dimensional XY model and Berezinskii–Kosterlitz–Thouless physics [13, 14, 15]. Extending the present framework to thermal states would make it possible to study how confinement diagnostics change with temperature and how finite-size effects smooth the transition on small lattices.

6.3 Adding fermionic matter

A third major direction is to include dynamical fermions. This would substantially enrich the accessible physics, allowing one to study screening, string breaking, flavor dependence, and more general phase structure in gauge theories with matter. It would also open the door to more specialized phenomena such as symmetric mass generation, where interactions gap fermions without producing a conventional symmetry-breaking bilinear condensate [16, 17].

6.4 Algorithmic improvements

Finally, the present implementation could be improved algorithmically by increasing the expressive power of the ansatz through deeper equivariant blocks or larger feature dimensions, and by improving the optimization procedure through better sampling, reduced autocorrelation in Metropolis-Hastings, and more advanced optimization methods such as stochastic reconfiguration or natural-gradient approaches [8, 18]. Such improvements may help explain the slower convergence observed at weaker coupling.

References

[1] D. Luo, S. Yuan, J. Stokes, and B. K. Clark, *Gauge equivariant neural networks for 2+1d u(1) gauge theory simulations in hamiltonian formulation*, arXiv preprint arXiv:2211.03198 (2022) , arXiv:2211.03198 [hep-lat].

[2] K. G. Wilson, *Confinement of quarks*, Physical Review D 10 no. 8, (1974) 2445–2459.

[3] J. B. Kogut, *An introduction to lattice gauge theory and spin systems*, Reviews of Modern Physics 51 no. 4, (1979) 659–713.

[4] M. Troyer and U.-J. Wiese, *Computational complexity and fundamental limitations to fermionic quantum monte carlo simulations*, Physical Review Letters 94 no. 17, (2005) 170201.

[5] E. Zohar, J. I. Cirac, and B. Reznik, *Quantum simulations of lattice gauge theories using ultracold atoms in optical lattices*, Reports on Progress in Physics 79 no. 1, (2016) 014401.

[6] J. Kogut and L. Susskind, *Hamiltonian formulation of wilson's lattice gauge theories*, Physical Review D 11 no. 2, (1975) 395–408.

[7] R. Orús, *A practical introduction to tensor networks: Matrix product states and projected entangled pair states*, Annals of Physics 349 (2014) 117–158.

[8] G. Carleo and M. Troyer, *Solving the quantum many-body problem with artificial neural networks*, Science 355 no. 6325, (2017) 602–606.

[9] M. Favoni, A. Ipp, D. I. Müller, and D. Schuh, *Lattice gauge equivariant convolutional neural networks*, Physical Review Letters 128 no. 3, (2022) 032003.

[10] J. Bender, P. Emonts, E. Zohar, and J. I. Cirac, *Real-time dynamics in compact qed using complex periodic gaussian states*, Physical Review Research 2 no. 4, (Oct., 2020) .

[11] L. G. Yaffe, *Confinement in lattice gauge theories*, Physical Review D 21 no. 6, (1980) 1574–1593.

[12] A. M. Polyakov, *Quark confinement and topology of gauge groups*, Nuclear Physics B 120 no. 3, (1977) 429–458.

[13] O. Borisenko, R. Fiore, M. Gravina, and A. Papa, *Critical behaviour of the compact 3d u(1) gauge theory at finite temperature*, arXiv preprint arXiv:1012.4942 (2010) , arXiv:1012.4942 [hep-lat].

[14] O. Borisenko, R. Fiore, M. Gravina, and A. Papa, *Deconfinement and universality in the 3d u(1) lattice gauge theory*, arXiv preprint arXiv:1507.00833 (2015) , arXiv:1507.00833 [hep-lat].

[15] B. M. Gripaios, *Variational analysis of deconfinement in compact u(1) gauge theory*, arXiv preprint hep-th/0211104 (2002) , arXiv:hep-th/0211104 [hep-th].

[16] Y.-H. Guo, X.-Y. Song, and C.-M. Jian, *Symmetric mass generation for kähler-dirac fermions*, arXiv preprint arXiv:2306.17420 (2023) , arXiv:2306.17420 [cond-mat.str-el].

[17] M. Golterman and Y. Shamir, *Propagator zeros and lattice chiral gauge theories*, arXiv preprint arXiv:2311.12790 (2023) , arXiv:2311.12790 [hep-lat].

[18] H. Lange, M. Reh, M. Schmitt, and F. Marquardt, *A review of neural quantum states*, arXiv preprint arXiv:2402.09402 (2024) , arXiv:2402.09402 [quant-ph].

A Details of code

In this section, we explain the codes that implement CNN and Metropolis-Hastings using the Pytorch library. This section focuses on programming, not physics.

A.1 ComplexCNN

```
1  class ComplexCNN(nn.Module):
2      def __init__(self, in_ch, hidden_ch, out_ch, kernel_size, n_layers=2):
3          super().__init__()
4          assert n_layers >= 1
5
6          layers = []
7          ch = in_ch
8          for _ in range(n_layers - 1):
9              layers += [ComplexConv2d(ch, hidden_ch, kernel_size), cELU()]
10             ch = hidden_ch
11         layers += [ComplexConv2d(ch, out_ch, kernel_size)]
12         self.net = nn.Sequential(*layers)
13
14     def forward(self, z):
15         return self.net(z)
```

- "nn.Sequential()" on Line 12 is a container from Pytorch that takes multiple nn.Modules and chains them together. It automatically feeds the output of one layer into the next. This is done by its own forward function:

```
def forward(self,x):
    for layer in self.layers:
    x = layer(x)
return x
```

- "*" in the argument on Line 12 is an unpacking operator, i.e. it removes the wrapping (brackets) from layers and feed only the contents inside to nn.Sequetial.
- self.net = nn.Sequential(*layer) makes the following net: input → first element in the layer → second element → ⋯ → output.
- The forward function returns self.net(z). self.net is an nn.Sequential object. When calling self.net(z), it automatically means self.net.forward(z).

A.2 Metropolis-Hastings

```
1  @torch.no_grad()
```

```python
def mh_step(model, theta, step_size=0.3):
    """
    One sweep: update each link once in random order.
    theta: (B,2,H,W)
    returns: (theta, acceptance_rate_float)
    """
    B, _, H, W = theta.shape
    n_links = 2 * H * W

    # log probability for |psi|^2
    logp = 2.0 * model(theta)  # (B,)

    perm = torch.randperm(n_links, device=theta.device)

    accepts = 0
    total = 0

    for idx in perm:
        dir_ = idx // (H * W)          # 0 or 1
        rem  = idx %  (H * W)
        y = rem // W
        x = rem %  W

        # propose random-walk update on this single link
        theta_prop = theta.clone()
        delta = (2.0 * torch.rand(B, device=theta.device, dtype=theta.dtype) - 1.0) * step_size
        theta_prop[:, dir_, y, x] = (theta_prop[:, dir_, y, x] + delta) % TWOPI

        logp_prop = 2.0 * model(theta_prop)  # (B,)

        # acceptance prob a = min(1, exp(logp_prop - logp))
        a = torch.exp(logp_prop - logp).clamp(max=1.0)

        u = torch.rand(B, device=theta.device, dtype=theta.dtype)
        acc = (u < a)  # (B,) boolean

        if acc.any():
            # accept only for the chains that passed
            theta[acc, dir_, y, x] = theta_prop[acc, dir_, y, x]
            logp[acc] = logp_prop[acc]

        accepts += int(acc.sum().item())
        total += B

    return theta, accepts / max(total, 1)


@torch.no_grad()
def mh_sample(model, theta, n_sweeps=10, step_size=0.3):
```

```
    accs = []
    for _ in range(n_sweeps):
        theta, acc = mh_step(model, theta, step_size=step_size)
        accs.append(acc)
    return theta, float(np.mean(accs))
```

Once we draw the wave function ψ_α from the model, we draw the angle configuration θ following the probability $|\psi_\alpha(\theta)|$.

1. Randomly sample one angular configuration θ using torch.rand.

```
theta = (TWOPI * torch.rand(batch, 2, L, L, device=device, dtype=dtype)) % TWOPI
# burn-in
theta, acc = mh_sample(model, theta, n_sweeps=burn_sweeps,
↪  step_size=mh_step_size)
```

2. Pick up a link randomly and make a small shift in the link angle. Let θ' be the new angle configuration. (Lines 26-28)

3. Accept the change if the new configuration is more probable in the proposed ground state by comparing $|\psi_\alpha(\theta')|$ and $|\psi_\alpha(\theta)|$. More specifically, we accept the new configuration when

$$\text{Accept when} \quad \frac{|\psi_\alpha(\theta')|^2}{|\psi_\alpha(\theta)|^2} > p_{random} \tag{50}$$

where $p_{random} \in (0, 1]$ is a randomly sampled acceptance threshold.

B Model Architecture

A compact $U(1)$ gauge field configuration can be represented in a perioidic $L \times L$ lattice (Fig 7), where $\theta_{v,\mu} \in [0, 2\pi)$ denotes the value of the gauge link angle originating from site v and pointing in the direction μ. A gauge field configuration, θ, which specifies the link angles on each edge of the lattice, has a shape of $(B, 2, L, L)$, where B is the batch size, 2 represents that μ can take on two values 1, and 2 corresponding to the x and y directions in figure 7 respectively and $L \times L$ represents the lattice dimensions.

The next step is to construct a neural network which takes as input the configuration θ and predicts the log amplitude of the wavefunction corresponding to that configuration, $\psi(\theta)$. The output of the network should be Gauge invariant i.e. $\psi(\theta)$ should be equal to $\psi(\theta')$ when $\theta'_{v,\mu} = \theta_{v,\mu} + \alpha(v+\hat{\mu}) - \alpha(v)$ (mod 2π), where $\alpha(v)$ is a real number assigned to the lattice site v and $\theta_{v,\mu} \in \theta$ and $\theta'_{v,\mu} \in \theta'$.

B.1 Constructing Gauge invariant objects

To transform to a suitable represention of the gauge fields, the link angles ($\theta_{v,\mu} \in [0, 2\pi)$) are converted into link variables (phases) $U_{v,\mu} = e^{i\theta_{v,\mu}}$, which makes it easier to construct Wilson plaquettes,

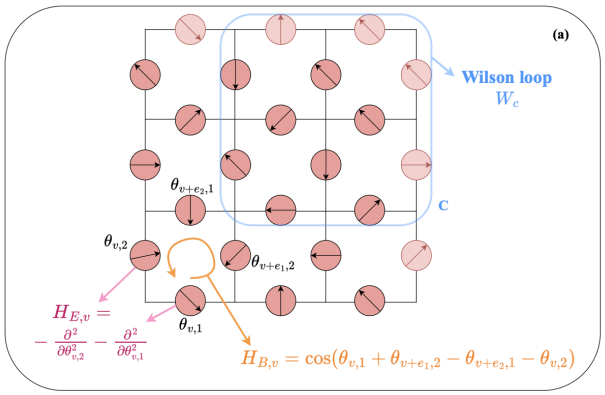

Figure 7: Lattice Configuration. (Image taken from [1].)

$W_\square = U_x(v)\, U_y(v+\hat{x})\, U_x^\dagger(v+\hat{y})\, U_y^\dagger(v)$, which remain invariant under a gauge transformation. Circular shifts and circular padding are also used to ensure that periodic boundary conditions are respected throughout the model.

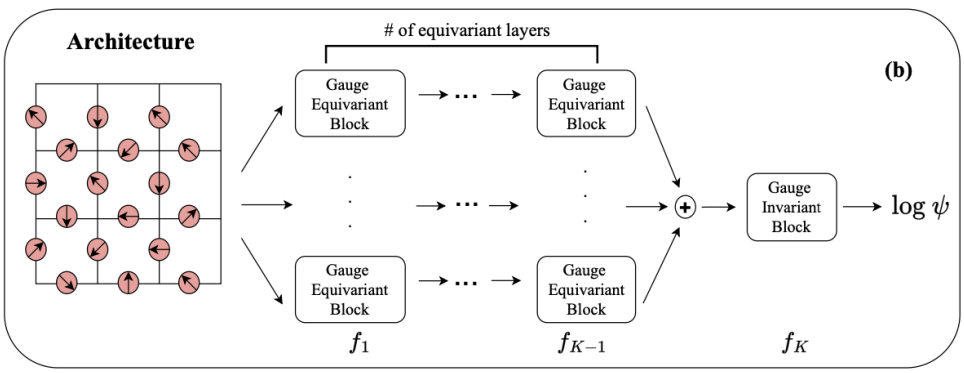

Figure 8: Model Overview. (Image taken from [1].)

B.2 Lift Layers

To enhance feature representations, the x and y-links are passed through separate 1×1 complex convolutions, producing multidimensional feature maps $\tilde{U}_x$ and $\tilde{U}_y$ which are stacked into link-like features $\tilde{U} \in \mathbb{C}^{B\times F\times 2\times L\times L}$. Keeping the two directions separate is essential as $\tilde{U}_x$ and $\tilde{U}_y$ play distinct geometric roles in the construction of gauge-invariant plaquettes .Making the $\tilde{U}$ values complex-valued is also important as the Gauge link variables are similar to phases on the unit circle, and are thus naturally represented and processed as complex numbers. An alternative way would be represent their real and imaginary parts separately, which would increase model complexity without utilizing the inherent physical properties.

B.3 Gauge Equivariant Layers

The workhorse of this model is a stack of gauge equivariant blocks (See Figure 9). In each block, the link variables are first converted to gauge invariant plaquettes,

$$P(v) = \tilde{U}_x(v)\,\tilde{U}_y(v+\hat{x})\,\tilde{U}_x^\dagger(v+\hat{y})\,\tilde{U}_y^\dagger(v),$$

upon which a complex CNN acts to produce a modulation field M. M is therefore gauge invariant by construction. Finally, the link variables are updated through the residual connection

$$\tilde{U} \leftarrow \tilde{U} + \tilde{U}\, c\mathrm{ELU}(M).$$

where $c\mathrm{ELU}(M)$ actiavation function is used to introduce non-linearity while minimizing vanishing gradients at $M < 0$, as $c\mathrm{ELU}(x) \neq 0$ when $x < 0$. The residual connection also allows to refine the links step by step instead of rewriting them from scratch, which preserves the geometrical structure of the links and ensures stability during training.

B.4 Gauge Invariant Layer

As the wavefunction is a scalar, the final layer of the model consists of a gauge invariant layer (Figure 10) which forms plaquette loops using the final link features ($\tilde{U}$), processes them through a complex CNN and averages over lattice sites to extract translation-invariant and gauge invariant summary features for configuration θ. These complex valued summary features are then split into real and imaginary parts and passed through a linear layer to output a single real number, which is the prediction for the log amplitude of the wavefunction, $\log\psi(\theta)$ for the gauge angle configuration θ. Predicting $\log\psi(\theta)$ instead of $\psi(\theta)$ compresses the output to a reasonable range and makes the calculation of ratios of the wavefunctions, such as $(\psi(\theta')/\psi(\theta))$ in the subsequent variational Monte-Carlo steps easier.

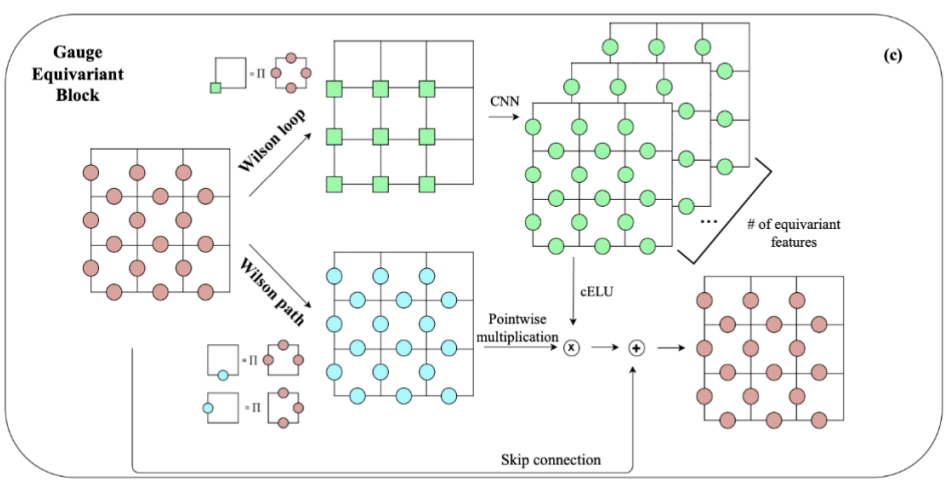

Figure 9: Gauge Equivariant Blocks. (Image taken from [1].)

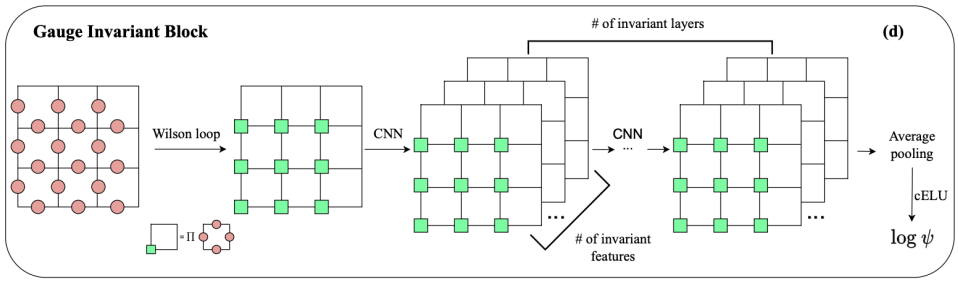

Figure 10: Gauge Invariant Layer. (Image taken from [1].)